

确保无缝转换：基于Fountain 1.1规范的YAML设计方案审查与验收标准

Fountain 1.1核心语法规则与行业实践基准

为了对YAML设计方案进行全面而精确的审查，首要任务是建立一个坚实且详尽的Fountain 1.1规范基准。Fountain作为一种旨在简化剧本创作流程的纯文本标记语言，其设计理念、核心语法以及随版本迭代而演进的规则，共同构成了评估任何中间表示（Intermediate Representation, IR）准确性的基石 [11](#) [26](#)。本章节将系统梳理Fountain 1.1的核心语法规则，并整合业界在特定剧本元素处理上的专业实践，为后续的审查工作提供权威的参照系。

Fountain的起源融合了Screenplay Markdown (SPMD) 和 Scrippets 两种早期规范，体现了社区协作与实用主义的特点 [10](#) [11](#)。其核心设计哲学是“简单、未来可读、易于存档”，旨在让编剧摆脱特定软件的束缚，使用任何文本编辑器进行创作，并通过辅助应用程序完成最终的排版和格式化 [11](#)。这种轻量级、开放式的特性使其成为剧本创作领域中Markdown的对应物，极大地促进了现代开发和协作工作流的集成 [26](#)。例如，Fountain脚本可以轻松地与GitHub等版本控制系统结合，实现高效的协同写作与历史追溯 [16](#)，甚至被用于训练大型语言模型（LLM）的优化任务中，作为标准化的中间表示（CIR）[4](#)。

Fountain 1.1的语法体系围绕一系列结构性元素构建，这些元素直接映射了传统剧本的组成部分。文件的开头通常是一个标题页，由一系列键值对构成，如 **TITLE:**、**AUTHOR:**、**CONTAC:** 和 **SOURCE:**，它们与正文之间以空行分隔 [2](#)。场景节（Scene Headings），通常被称为INT./EXT.或日/夜标记，是组织故事结构的基本单元，它们如同文档的章节标题，帮助编剧和制作团队理解场景的转换 [47](#)。动作（Action）元素是一段或多段描述性文字，用于描绘场景环境、角色行为以及其他视觉信息。当角色准备说话时，其姓名以大写形式独占一行，形成角色名（Character）元素，其下方紧随的文本则构成对话（Dialogue）元素 [2](#)。此外，括号内容（Parentheticals）用于在角色名或对话之后添加简短的补充说明，指示角色的语气或动作，通常用括号（ ）包裹 [24](#)。

Fountain 1.1版本的重大更新引入了“显式强制”（Use The Force）主题，显著增强了作者对格式的控制力，这对于评估YAML方案的兼容性至关重要 [1](#)。其中一项关键改进是允许通过在行首添加 @ 符号来强制指定一个角色名。这一机制解决了处理混合大小写名称（如 @McCLANE）或非罗马字符名称（如 @黒澤）的难题，确保解析器能将其正确识别为

角色名，而非意外的动作或对话块 1。另一项重要变更涉及动作块的强制。旧版本中使用的、通过在行尾添加两个不可见空格来强制动作块的方法，因其不易察觉且容易出错而被废弃 1。Fountain 1.1推荐使用在动作行之前添加！符号的新方法，该符号会被解析器移除，从而明确地将整块文本定义为动作 1。此外，Fountain 1.1还引入了一个全新的Lyrics元素，专门用于处理歌曲歌词。通过在新行前加上波浪符~，作者可以声明该行是歌词，解析器会移除该符号，仅保留内容，以便应用程序对其进行特殊样式化（例如，在屏幕上显示为斜体）1。值得注意的是，Fountain 1.1还放宽了对角色扩展（即与角色名在同一行的括号内容）大小写的限制，现在无论其大小写如何，解析器都能正确地将其解释为角色元素的一部分 1。

除了语法层面的演进，业界在实际应用中也形成了许多不成文但至关重要的最佳实践。一个典型的例子是关于(CONT'D)标签的处理。行业内普遍认为，软件不应自动向剧本中插入(CONT'D)标签 12。这是因为自动化的(CONT'D)可能会错误地违背作者的意图。例如，在电影《地心引力》的一个场景中，女主角在经历了长时间的沉默后才开口，如果由软件自动生成，可能会错误地在其台词前加上(CONT'D)，从而误导读者 12。正确的做法是，软件可以智能地为手动开始输入(CONT'D')的作者提供自动补全功能，但绝不应自行插入 12。这背后体现了一种“作者优先”的原则，即尊重并保持作者对作品的绝对控制权。与此相对的是页面断行指示符（如MORE），这类标识是在打印或预览时由软件动态添加的，用于指示内容跨越页面，它们并非源文本的一部分，因此不应被视为作者意图的表达 12。一个优秀的YAML方案必须能够理解和表达这种细微的差别，即(CONT'D')是一个需要被精确存储和回显的作者输入，而不是一个需要程序动态生成的元素。综上所述，Fountain 1.1不仅在语法上提供了更强的灵活性和控制力，其生态也围绕着尊重作者意图这一核心理念发展。这些规则和实践共同构成了评估YAML设计方案是否合格的黄金标准。

Fountain 1.1 核心元素	描述	强制语法 (Fountain 1.1)
角色名 (Character)	角色的名字，通常为大写，独占一行。	使用 @ 前缀强制指定，即使名字不符合常规大写格式 1。
动作 (Action)	描述场景、角色动作等的段落文本。	使用 ! 前缀强制指定单行或多行文本为动作块 1。
歌词 (Lyrics)	歌曲的歌词文本。	使用 ~ 前缀标记，内容将被特殊样式化 1。
角色扩展 (Parenthetical)	描述角色说话方式的补充说明，通常在角色名或对话后。	不再强制要求大写 1。
(CONT'D) 标签	标识角色在短暂停顿后继续说话。	作者手动输入，软件可提供自动补全，但禁止自动插入 12。

YAML设计方案的结构合规性与合理性评估

在确立了Fountain 1.1的详细规范作为评估基准后，下一步是对所提供的YAML设计方案进行严格的结构合规性审查。用户的验收标准明确指出，该YAML设计不仅要作为高效的中间表示，更要成为一个易读、准确、确定、完整且合理的最终输出产物。这意味着其结构设计必须在技术严谨性与人类友好性之间取得精妙的平衡，避免因过度工程化或晦涩的抽象而损害其作为剧本载体的根本价值。

首先，易读性是该YAML方案能否作为最终产物的关键考量。一个可供人类专家审阅和编辑的YAML文件，其结构应当直观且命名清晰，能够让熟悉剧本格式的人快速理解剧本的层级关系和内容构成。借鉴Elasticsearch中 `_source` 字段的概念，一个理想的YAML结构应能忠实地反映源文档的逻辑和物理布局，使开发者或编辑者能够轻易地从YAML回溯到原始的Fountain文本 [20](#)。这意味着，设计应优先采用扁平化的序列化结构，即将剧本中的所有元素——无论是标题页、场景节还是对话——按照它们在源文件中出现的顺序，组织在一个顶层数组中。这种线性结构最贴近Fountain这种源于纯文本的格式的本质，避免了不必要的树状或图状嵌套，除非有非常强的理由（例如，管理复杂的跨元素依赖关系）。每个元素都应由一个独立的对象表示，并通过一个明确的类型字段（如 `type: 'character'`）来区分其身份，而不是依赖对象的属性集合来隐式推断。

其次，准确性是确保数据无损传递的基础。YAML模型中的每一个字段都必须拥有明确定义的数据类型（字符串、数字、布尔值、嵌套对象等）和约束。例如，场景节中的场内/场外（INT/EXT）和日/夜（DAY/NIGHT）属性，不应是任意字符串，而应是受控的枚举类型，以防止无效或拼写错误的数据被接受。同样，日期和时间戳等字段应有严格的格式要求。Pydantic等验证框架的应用正是为了实现这一点，它可以在数据注入时进行静态类型检查和验证，从而在源头杜绝错误数据，保证YAML实例的内在一致性 [42](#)。这种严格性对于一个既要被机器处理又要供人阅读的格式来说尤为重要，因为它减少了歧义，提高了系统的健壮性。

第三，确定性是双向转换能力的保证。对于完全相同的Fountain源文本，转换生成的YAML IR必须是完全一致的，不能有任何随机因素或依赖外部状态的转换逻辑。这意味着解析器的行为必须是完全可预测和可重复的。例如，在处理多行动作块时，其内部换行符的处理方式必须统一且固定。如果一个YAML方案在多次转换同一份Fountain文件时会产生不同的YAML输出，那么它就无法满足“往返转换”的基本要求，因为下游系统可能无法稳定地处理这种不一致的IR。因此，在审查方案时，必须关注其解析算法的确定性，确保没有隐藏的状态机或依赖于环境变量的逻辑。

第四，完整性是评估兼容性的前提。该YAML方案必须能够表示Fountain 1.1所定义的全部语法元素，包括但不限于标题页、场景节、动作、角色、对话、括号内容，以及Fountain 1.1特有的强制语法（`@`, `!`, `~`）和新增的Lyrics元素。任何遗漏都将直接导致信息丢失，

破坏双向转换的能力。例如，如果方案没有为强制角色名 (@...) 或强制动作块 (!...) 设计专门的标识字段，那么在从YAML转换回Fountain时，这些信息就会丢失，导致生成的Fountain文本在格式上与原稿不一致。同样，对于Lyrics元素的支持也必须是显式的，需要有一个字段来区分它与普通动作文本。

最后，合理性与业界习惯是判断方案成熟度的试金石。设计方案应避免不必要的复杂性，其结构应尽可能地与Fountain的线性、自上而下的叙事流保持一致。例如，是否需要为每个元素创建一个独立的顶层对象，还是更合理的做法是将所有剧本元素统一在一个序列化的列表中？业界最佳实践倾向于后者，因为它更符合文本文件的自然顺序。此外，方案应考虑未来的可扩展性，例如，为可能的Fountain新版本或自定义语法预留扩展点，如一个通用的custom_data或metadata对象，用于存储不在标准Schema中的额外信息。这不仅能增加方案的灵活性，也符合现代软件架构中对松耦合和开放-封闭原则的追求。通过对这些维度的综合评估，我们可以判断该YAML设计方案是否真正达到了用户所期望的高标准。

与Fountain 1.1的双向兼容性 & 信息保真度审查

双向兼容性是本次审查的核心焦点，它直接决定了该YAML设计方案能否作为一个可靠的中间表示（IR）。一个成功的方案必须能够在Fountain 1.1源文件与YAML IR之间实现“零信息损失”的来回转换。这意味着，从Fountain源文件A转换而来的YAML IR，再经过反向转换生成的目标Fountain文件B，其内容应与原始文件A在语义和格式上完全一致。任何微小的差异，无论是丢失的标点符号、错误的大小写，还是丢失的强制语法标记，都可能在协作或自动化流程中引发问题，因此必须予以严格杜绝。

审查的重点在于方案对Fountain 1.1特有语法元素的支持程度。Fountain 1.1引入的“显式强制”机制是提升作者控制力的关键，也是衡量兼容性的试金石 ¹。方案必须为以下三种强制语法提供明确且无歧义表示：1. 强制角色名 (@)：方案的Character对象模型必须包含一个布尔型字段（例如 is_explicitly_forced 或类似的命名），用于标识该角色名是否由 @ 符号强制指定。在反向转换时，若此字段为真，则必须在生成的Fountain文本中相应地添加 @ 符号。2. 强制动作块 (!)：同样，ActionBlock或等效对象模型需要一个布尔型字段来标记其是否由 ! 强制。反向转换时，该标志位应触发 ! 符号的添加。3. 歌词 (~)：由于Fountain 1.1引入了新的Lyrics元素，方案必须能够区分它与普通的Action或Dialogue。这可以通过为Line或基础文本对象设置一个type字段来实现，其值可以是 'action', 'dialogue', 或 'lyric'。当type为 'lyric' 时，反向转换必须在对应的Fountain行首添加 ~ 符号。

除了对新语法的支持，方案还必须妥善处理Fountain 1.1带来的其他变化，例如角色扩展不再强制大写的要求 [1](#)。虽然这看似是一个较小的改动，但它意味着YAML模型中的Parentetical对象不应再对其内容的大小写施加约束，以确保原始作者的意图得以保留。

为了验证双向转换的可靠性，必须建立一套端到端的自动化回归测试。这套测试应包含一个覆盖面广泛的测试用例集，涵盖Fountain 1.1的各种语法场景，包括：

- 标准的剧本结构（标题页、场景节、动作、对话）。
- 所有Fountain 1.1的强制语法（`@`，`!`，`~`）。
- 各种括号内容和角色扩展的组合。
- 边缘情况，例如空的动作块、连续的对话、以及包含特殊字符或非罗马字符的文本。

测试流程应为：对于每一个测试用例（一个Fountain文件），执行“Fountain -> YAML -> Fountain”的转换链，然后将最终生成的Fountain文件与原始输入文件进行逐字比较。理想情况下，两者应完全相同。任何差异都必须被记录和修复。这种严格的“往返转换”测试是确保信息保真度最有效的方法。

此外，业界实践也为我们提供了宝贵的参考。关于(CONT'D)标签的处理就是一个绝佳的例子。行业共识是软件不应自动插入(CONT'D)，而应只在作者手动输入时提供补全 [12](#)。因此，YAML方案对此类标签的处理策略必须与此原则保持一致。它应该将作者手动输入的(CONT'D)视为一个普通的、需要被精确存储和回显的文本片段，而不是一个需要程序逻辑去动态生成的特殊元素。如果方案在设计上试图“智能”地处理(CONT'D)，比如根据前后文自动判断并添加，那么它就是与业界最佳实践相悖的，可能会在实际应用中引起作者的反感。一个合格的方案应该忠实于作者的每一次手动输入，即使是像(CONT'D)这样的标签。

最后，对于那些Fountain规范本身并未明确定义的、但在高级应用中可能出现的格式，如“双人对话”（Dual Dialogue），YAML方案应表现出合理的灵活性 [25](#)。它不应僵化地拒绝此类非标准用法，而应通过提供扩展机制（如前述的custom_data字段）来容纳它们，或者至少在文档中清晰地说明其处理此类情况的默认策略。这种前瞻性设计能够极大地增强方案的适应性和生命力，使其不仅仅局限于Fountain 1.1的官方规范之内。

追溯性机制完整性与应用价值分析

追溯性是现代软件工程和需求管理中的一个核心概念，它指的是能够清晰地追踪一个系统组件与其上游来源（如需求文档、设计规格）之间的关系 [22](#) [23](#) [29](#)。在当前的审查背景下，追溯性机制指的是在YAML中间表示中，为每一个原子元素附加元数据，使其能够精确地追溯到其在原始Fountain文本中的确切位置和上下文。这个source_ref机制的完整性，直接决定了该YAML方案的应用上限和潜在价值。

之所以强调追溯性的重要性，是因为该YAML方案被定位为一个既是中间表示又是最终输出产物的设计 [4](#)。如果它仅仅在内部被机器处理，那么source_ref或许只是一个锦上添花的功能。然而，一旦它成为人类可以直接查看、编辑和分发的最终产物之一，其缺乏追溯能力的弊端便会凸显。想象一个场景：一位编剧收到一个同事修改过的YAML剧本文件。如果某个对话段落出现了格式错误，编剧很难在YAML中定位到问题究竟出在原始Fountain文件的哪一行、哪一列。一个带有完善source_ref的方案则能解决这个问题，它能让编辑工具高亮显示有问题的原始文本行，极大地简化了调试和协作流程。

一个完整的source_ref机制应能为YAML IR中的每一个原子元素提供丰富的溯源信息。这不仅仅是为一个大的元素块（如整个场景）打上标记，而是要深入到最小的文本单位。一个理想的source_ref对象至少应包含以下字段：

- ``line_number``：元素在原始Fountain文本文件中的行号（从1开始计数）。
- ``char_offset``：元素在所属行内的字符偏移量，精确到具体是哪一个字符。
- ``original_text``：元素在原始文件中对应的完整文本片段。这对于需要对比修改前后内容的场景至关重要。
- ``parent_element_id``：如果YAML结构是分层的，该字段可用于指向其逻辑上的父元素，从而重建文档的树状结构。

检验该机制完整性的关键是看它是否覆盖了IR中的每一个原子元素。例如，一个Dialogue对象不仅仅是一个包含文本的字符串，它应该是一个包含了text字段和source_ref对象的复合对象。同样，一个ActionBlock中的每一行文本，如果被单独建模，也都应该拥有自己的source_ref。如果一个source_ref缺失，或者其信息不准确（例如，行号计算错误），那么整个追溯链条就会中断，方案的应用价值将大打折扣。

具备完善追溯性机制的YAML方案，其潜力远超一个简单的转换器。它能支撑起一系列更复杂的、基于AI或自动化的工作流：1. 智能版本比对与合并：在协同写作中，当多个编剧同时修改同一个脚本时，传统的文本比对工具难以理解剧本的结构。一个带有source_ref的YAML方案可以实现结构化的差异分析，例如，清晰地指出“A编剧修改了第15场景的动作描述，B编剧修改了第15场景中John的角色名”。这为解决版本冲突提供了前所未有的便利。2. 基于AI的剧本分析与建议：AI模型可以利用source_ref信息，

在发现潜在问题（如角色出场频率过高、场景转换突兀）时，直接在原始剧本的相应位置提出修改建议，而不是给出模糊的指导。

3. 与在线协作平台的深度集成：在Obsidian等笔记应用中，插件可以利用source_ref信息，将YAML中的剧本元素与文档中的特定区块绑定，实现真正的实时双向同步和交互式预览 [17](#)。

4. 自动化审计与合规检查：在大型制片项目中，可以编写脚本来遍历整个YAML剧本库，利用source_ref信息快速定位所有不符合特定格式规范（如公司内部标准）的段落，并生成详细的审计报告。

总之，追溯性机制并非一个孤立的技术细节，而是决定该YAML方案能否从一个简单的“翻译器”升级为一个强大“生态系统基座”的关键。一个设计良好的source_ref机制，能够将剧本的静态内容与动态的上下文和位置信息绑定在一起，解锁无数高级应用场景，为创作者、制片方和开发者提供前所未有的效率和控制力。

综合洞察、验收标准与战略建议

经过对Fountain 1.1核心语法规则、YAML设计方案的结构合规性、双向兼容性及追溯性机制的系统性审查，本报告现提出综合性的洞察、一套明确的验收标准，并给予战略性的建议。此次审查不仅是对一个技术方案的评估，更是对其是否能满足真实业界专业需求、符合行业习惯的深度拷问。

核心洞察

第一，Fountain 1.1规范本质上是一个“可控性”不断增强的演进过程。相较于早期版本，Fountain 1.1更加注重赋予作者对剧本格式的绝对控制权，这主要通过@、!、~等“显式强制”语法得以实现 [1](#)。因此，任何声称兼容Fountain 1.1的YAML方案，都必须将对这些“强制”机制的支持置于最高优先级。忽视这一点，将直接导致方案在核心兼容性上存在致命缺陷。

第二，将YAML设计为一个“最终输出产物”带来了双重挑战。一方面，它需要是机器高度优化的中间表示（IR），结构清晰、类型安全；另一方面，它又必须是人类可读的，其结构应直观、命名应清晰，便于人工审阅和调试。这对YAML的结构设计提出了极高的要求，要求设计者在技术严谨性和人类友好性之间取得精妙的平衡。过于复杂的嵌套结构或晦涩的命名约定会严重损害其作为“最终产物”的可用性。

第三，追溯性（traceability）机制是决定方案上限的关键因素。一个带有完善source_ref机制的YAML方案，其潜力远大于一个没有此机制的方案。前者不仅能实现基本的、无损的文本转换，更能支撑起更复杂的生态系统，如基于人工智能的剧本分析、

自动化的版本冲突解决、与在线协作平台的深度集成等 62 64。缺乏细粒度的追溯能力，将极大地限制方案在高级应用场景中的价值。

第四，业界实践提供了宝贵的“隐性”标准，这些标准超越了纯粹的语法范畴，触及了创作伦理和技术哲学。关于(CONT'D)标签“不自动插入”的讨论，深刻揭示了“作者优先”这一核心原则 12。一个好的YAML方案不仅要能转换语法，更要能表达和尊重作者的意图。它应该能够忠实地存储作者的每一次手动输入，即使是像(CONT'D)这样的标签，也应被视为作者意图的直接体现，而非需要程序逻辑去“纠正”或“优化”的对象。

可操作的验收标准

为了给用户一份具有明确指导意义的验收清单，我们提出以下基于上述分析的具体、可衡量的检查项。通过逐一核对这些标准，可以系统性地评估该YAML方案的质量。

检查维度	检查项	预期结果/标准
结构合规性	根对象结构	YAML根对象是否为单一对象，且包含一个名为 script_elements 的数组，其中按序排列所有剧本元素？
	元素对象定义	是否存在清晰的、用于定义不同元素类型的顶层对象（如 TitlePage, SceneHeading, Action, Character, Dialogue, Lyric, Parenthetical）？
	字段命名一致性	元素对象的字段命名是否遵循了统一的风格（如蛇形命名法 element_type）？
Fountain 1.1兼容性	强制角色名	Character 对象是否包含一个布尔型字段（如 is_explicitly_forced）来标识 @ 的使用？
	强制动作块	ActionBlock 对象是否包含一个布尔型字段（如 is_explicitly_forced）来标识 ! 的使用？
	歌词元素	Line 或基础文本对象是否有 type 字段（或类似机制），用于区分普通动作、对话和由 ~ 标识的 Lyrics？
	往返转换测试	是否存在端到端自动化测试，验证包含所有Fountain 1.1语法的复杂剧本，在YAML和Fountain间来回转换后，输出的Fountain文本与输入的完全一致（逐字比较）？
追溯性 (source_ref) 完整性	原子元素覆盖	每个原子元素对象（如每一条对话、每一个动作行）是否都包含一个 source_ref 对象？
	source_ref 字段	source_ref 对象是否至少包含 line_number 和 char_offset 两个字段？
	信息准确性	转换过程是否保证了 source_ref 信息的准确计算和传递？
合理性与扩展性	结构合理解释	方案文档是否解释了为何采用当前的序列化列表结构，而不是其他可能的结构？
	未来扩展性	方案是否为未来的扩展（如新的Fountain版本、自定义语法）留下了清晰的接口或注释字段（如 custom_data）？
	引用链接正确性	文档中所有引用的链接（如Fountain.io官方语法指南）是否都已核实并正确无误？

战略建议

建议采纳一套分阶段的实施与验收策略。第一阶段，集中资源解决最关键的兼容性问题，特别是对Fountain 1.1强制语法和Lyrics元素的支持，并立即着手搭建全面的往返转换回归测试套件。这是方案能否成立的生命线。第二阶段，在确保核心兼容性的基础上，完善YAML的结构设计，提升其易读性和人类友好性，并正式引入和完善source_ref追溯性机制。第三阶段，基于一个稳固、可靠的IR，探索和构建基于此方案的上层应用和服务，如Web编辑器、协作平台、AI分析工具等，从而最大化该方案的价值。

最终，对这份YAML设计方案的审查是一项系统工程，它要求审查者同时扮演“语言学家”、“架构师”和“质量工程师”的多重角色。最终的验收报告不应只给出“通过”或“不通过”的结论，而应基于本报告中详尽的维度和标准，提供一份充满洞察力的反馈，明确指出方案的优势所在、潜在的风险以及具体的、可操作的改进建议。

参考文献

1. Fountain 1.1: Use The Force - John August <https://johnaugust.com/2014/fountain-1-1-use-the-force>
2. Fountain Screenplay Syntax Guide | PDF | Bracket - Scribd <https://www.scribd.com/document/700794159/Syntax-Fountain>
3. Fountain Conversion within a document? - Literature & Latte Forums <https://forum.literatureandlatte.com/t/fountain-conversion-within-a-document/26865>
4. Canonical Intermediate Representation for LLM-based optimization ... <https://arxiv.org/abs/2602.02029>
5. [PDF] IRCoder: Intermediate Representations Make Language Models ... <https://aclanthology.org/2024.acl-long.802.pdf>
6. Intermediate Performance Oriented Fountain Codes With Feedback https://www.researchgate.net/publication/344142977_REF_Codes_Intermediate_Performance_Oriented_Fountain_Codes_With_Feedback
7. Improved online fountain codes | Semantic Scholar <https://www.semanticscholar.org/paper/f7d7f1165e15405ee92175925541ce086dafdda4>
8. (PDF) A Guide to Using GitHub for Developing and Versioning Data ... <https://www.researchgate.net/publication/>

353749417_A_Guide_to_Using_GitHub_for_Developing_and_Versioning_Data_Standards_and_Reporting_Formats

9. [PDF] What's Inside a GitHub Repository? An Empirical Study on ... - arXiv <https://arxiv.org/pdf/2605.16701>
10. SPMD is now Fountain - Prolost <https://prolost.com/blog/2012/2/8/spmd-is-now-fountain.html>
11. Introducing Fountain - John August <https://johnaugust.com/2012/introducing-fountain>
12. Fountain | johnaugust.com <https://johnaugust.com/apps/fountain>
13. How to use floating number in yaml file used in Git Actions <https://stackoverflow.com/questions/73608503/how-to-use-floating-number-in-yaml-file-used-in-git-actions>
14. english-words.txt - Miller <https://miller.readthedocs.io/en/6.13.0/data/english-words.txt>
15. words-333333 - cs.Princeton <https://www.cs.princeton.edu/courses/archive/spring18/cos226/assignments/autocomplete/testing/words-333333.txt>
16. Versioning comic scripts using Fountain and GitHub | Scott Lee Chua <https://scottleechua.com/blog/versioning-comic-scripts-using-fountain-and-github/>
17. Fountain — write screenplays in obsidian - Share & showcase <https://forum.obsidian.md/t/fountain-write-screenplays-in-obsidian/113893>
18. 什么是BNF（巴科斯范式）和EBNF - 知乎专栏 <https://zhuanlan.zhihu.com/p/1887806362144199098>
19. BNF and EBNF Grammar Overview | PDF | Computer Science - Scribd <https://www.scribd.com/document/627589103/bnf-and-rules>
20. _source field | Elasticsearch Reference <https://www.elastic.co/docs/reference/elasticsearch/mapping-reference/mapping-source-field>
21. The fountain model and its impact on project schedule <https://www.semanticscholar.org/paper/The-fountain-model-and-its-impact-on-project-Pillai/aadb03a25265deff51035a6ac9d4ef0f87ec5c3b>
22. Recovering Semantic Traceability between Requirements and ... https://www.researchgate.net/publication/359175124_Recovering_Semantic_Traceability_between_Requirements_and_Source_Code_Using_Feature_Representation_Techniques
23. Traceability and reuse mechanisms, the most important properties of ... <https://link.springer.com/article/10.1007/s10664-023-10428-2>
24. Help with nested styles and line break - Adobe Community <https://community.adobe.com/questions-671/help-with-nested-styles-and-line-break-888420>

25. Dual Dialogue - Wish List - Literature & Latte Forums <https://forum.literatureandlatte.com/t/dual-dialogue/4884>
26. Why I like writing in Fountain - John August <https://johnaugust.com/2013/why-i-like-writing-in-fountain>
27. Enhancing Traceability Link Recovery with Fine-Grained Query ... <https://www.mdpi.com/2078-2489/14/5/270>
28. Field Extraction for Source Rules - AppDynamics SaaS - Splunk Help <https://help.splunk.com/en/appdynamics-saas/analytics/26.5.0/analytics/configure-analytics/collect-log-analytics-data/configure-log-analytics-using-source-rules/field-extraction-for-source-rules>
29. (PDF) Enhancing Traceability Link Recovery with Fine-Grained ... https://www.researchgate.net/publication/370508827_Enhancing_Traceability_Link_Recovery_with_Fine-Grained_Query_Expansion_Analysis
30. FOUNTAIN: a modular research platform for integrated real-world ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC11445988/>
31. Web of Things (WoT): Use Cases and Requirements - W3C <https://www.w3.org/TR/wot-usecases/>
32. AkShare 股票数据 - 书栈网 <https://www.bookstack.cn/read/akshare/43c9e1e9f815ea0e.md>
33. HRM (@public.health.md) - TikTok <https://www.tiktok.com/@public.health.md>
34. ManipArena: Comprehensive Real-world Evaluation of Reasoning ... <https://arxiv.org/html/2603.28545v1>
35. Advancing Open-source World Models - arXiv <https://arxiv.org/html/2601.20540v1>
36. 1 Introduction - arXiv <https://arxiv.org/html/2510.20790v1>
37. [PDF] Development of Quantum InterConnects (QuICs) for Next ... - arXiv <https://arxiv.org/pdf/1912.06642>
38. [PDF] Roadmap towards the redefinition of the second - arXiv <https://arxiv.org/pdf/2307.14141>
39. Llasa: Scaling Train-Time and Inference-Time Compute for Llama ... <https://arxiv.labs.arxiv.org/html/2502.04128>
40. LLM Translation of Compiler Intermediate Representation - arXiv <https://arxiv.org/abs/2605.08247>
41. Can Large Language Models Understand Intermediate ... <https://openreview.net/forum?id=zDieh7VWfN>

42. Why We Need IR (Intermediate Representations) for LLM Workflows <https://www.linkedin.com/pulse/compiler-prompts-why-we-need-ir-intermediate-llm-workflows-gaur-kr4cc>
43. OLEVS 6728 Trend Fashion Men's Watches Skeleton Transparent ... <https://www.aliexpress.com/item/1005009544624036.html>
44. twitter user gender classification nlp - Kaggle <https://www.kaggle.com/code/qredness/twitter-user-gender-classification-nlp>
45. Mountain, a Tool for Screenwriting with Fountain | mjrusso.com <https://mjrusso.com/mountain-screenwriting-with-fountain/>
46. Import Plain Text Formatted Screenplay Fountain not being formatted <https://forum.literatureandlatte.com/t/import-plain-text-formatted-screenplay-fountain-not-being-formatted/45121>
47. Outlining scripts in Fountain - John August <https://johnaugust.com/2013/outlining-scripts-in-fountain>
48. Atomic Clock Ensemble in Space - arXiv <https://arxiv.org/html/2411.02912v1>
49. Testing screened scalar-tensor theories of gravity with atomic clocks <https://arxiv.org/html/2410.17292v2>
50. GeCo: A Differentiable Geometric Consistency Metric for Video ... <https://arxiv.org/html/2512.22274v1>
51. [PDF] Circulating pulse cavity enhancement as a method for ... - arXiv <https://arxiv.org/pdf/2012.06426>
52. [PDF] Operationalization of Relativistic Motion - arXiv <https://arxiv.org/pdf/1205.2680>
53. [PDF] TAROT: Towards Optimization-Driven Adaptive FEC Parameter ... <https://arxiv.org/pdf/2602.09880>
54. [PDF] Model-Native Computing Architecture - arXiv <https://arxiv.org/pdf/2606.00288>
55. [PDF] Structure-Preserving Error-Correcting Codes for Polynomial Frames <https://arxiv.org/pdf/2510.13882>
56. [PDF] METTLE: Efficient Streaming Erasure Code with Peeling Decodability <https://arxiv.org/pdf/2602.10020>
57. Debian -- Source Packages in "bullseye" <https://packages.debian.org/source/bullseye/allpackages>
58. <https://zenodo.org/records/6000983/files/Custom-La...> <https://zenodo.org/records/6000983/files/Custom-Labels.csv?download=1>
59. <https://packages.debian.org/source/bullseye/allpac...> <https://packages.debian.org/source/bullseye/allpackages?format=txt.gz>

60. ubuntupackages.cgi - Ultimate Debian Database <https://udd.debian.org/cgi-bin/ubuntupackages.cgi>
61. Spacemacs layers list <https://www.spacemacs.org/layers/LAYERS.html>
62. Speed up Functional Specs and Traceability (Without Excel or Scripts) <https://community.atlassian.com/forums/App-Central-articles/SDLC-in-Atlassian-Speed-up-Functional-Specs-and-Traceability/ba-p/3054530>
63. Requirement Traceability Report - TestFLO - Test Management for Jira <https://deviniti.com/support/addon/server/testflo-92/latest/requirement-traceability-report/>
64. Development Plan - Traceability - StrictDoc Documentation https://strictdoc.readthedocs.io/en/stable/stable/docs/strictdoc_24_development_plan-TRACE.html
65. Traceability in IBM Engineering Requirements Management DOORS <https://www.ibm.com/docs/en/engineering-lifecycle-management-suite/doors/9.7.1?topic=requirements-links-traceability>
66. [PDF] EBNF: A Notation to Describe Syntax <https://www.cs.cmu.edu/~pattis/misc/ebnf2.pdf>
67. Usage Guide — FDB 2.0.3 documentation <https://fdb.readthedocs.io/en/latest/usage-guide.html>
68. [XLS] Requirements Traceability Matrix https://fdotwww.blob.core.windows.net/sitefinity/docs/default-source/it/pdm/pdm2/activetemplates/requirements-traceability-matrix.xlsx?sfvrsn=c3c276f1_2
69. [PDF] Functional Specification Document https://www.esma.europa.eu/sites/default/files/library/esma65-8-9568_-_firds_reference_data_functional_specifications_v2.10.pdf